

Rapport

Analyse Syntaxique Projet

Membres du groupe :

Moukheiber Karl

Tran Phan

Groupe 1

Table des matières

3	Présentation
3-7	Choix
8	Difficultés
8	Tests
9	Conclusion

Présentation

Le rapport expliquera les choix qu'on a fait, dans l'écriture de l'analyseur syntaxique avec flex et bison, et les difficultés rencontrées.

Le projet consiste à construire un analyseur syntaxique pour TPC, un sous-ensemble du C. Quand un fichier .tpc passé en entrée ne contient pas d'erreur syntaxique ou lexicale, le programme le traduit en arbre abstrait.

Pour compiler le programme : **make**

Pour effacer les fichiers produits par le makefile : **make clean**

Pour exécuter le programme depuis la racine du projet (-h pour afficher l'aide, -t pour afficher l'arbre abstrait) :

`./bin/tpcas [OPTIONS...] < fichier.tpc`

Choix

```
%union {
    char byte;
    int num;
    char ident[256];
    char type[10];
    char comp[3];
    Node *node;
}

%token <byte> CHARACTER ADDSUB DIVSTAR
%token <num> NUM
%token <ident> IDENT
%token <type> TYPE
%token <comp> ORDER EQ
%token OR AND IF ELSE WHILE RETURN VOID STRUCT
```

Tout d'abord, voici ci-dessus tout ce que le lexer peut prendre. Il peut repérer un caractère, un nombre, un identifiant (on a fixé sa longueur à 256), un type (sachant qu'on a seulement **int** et **char** en type, on peut se permettre de mettre la longueur à 10), ou un opérateur de comparaison ou d'égalité.

Ensuite, pour l'extension de la grammaire, afin qu'il autorise les types structurés, on a bien modifié la grammaire existante.

```
DeclVars:
  DeclVars TYPE Declarateurs ';' { Node *decl = makeNode(declVar);
    if (strcmp($2, "int") == 0) addChild(decl, makeNode(typeInt));
    else if (strcmp($2, "char") == 0) addChild(decl, makeNode(typeChar));
    addChild(decl, $3);

    addChild($1, decl);
    $$ = $1;
  }

  | DeclVars STRUCT IDENT Declarateurs ';' { Node *decl = makeNode(declVarStruct);

    addChild(decl, makeNode(typeStruct));
    addChild(decl, makeNode(id));
    addChild(decl, $4);
    addChild($1, decl);

    $$ = $1;
  }

  | DeclVars STRUCT IDENT CorpsStruct ';' { Node *decl = makeNode(declVarStruct);

    addChild(decl, makeNode(typeStruct));
    addChild(decl, makeNode(id));
    addChild(decl, $4);
    addChild($1, decl);

    $$ = $1;
  }

  | { $$ = makeNode(declVars); }
;
```

Lors de la déclaration des variables globales et locales ci-dessus, on a ajouté la déclaration des types structurés et des variables d'un type structuré.

```
CorpsStruct: '{' DeclStruct '}' { $$ = $2; }
;
```

```
DeclStruct:
  DeclStruct TYPE Declarateurs ';' { Node *n = makeNode(declChamps);
    if (strcmp($2, "int") == 0) addChild(n, makeNode(typeInt));
    else if (strcmp($2, "char") == 0) addChild(n, makeNode(typeChar));
    addChild(n, $3);

    addChild($1, n);
    $$ = $1;
  }

  | DeclStruct STRUCT IDENT Declarateurs ';' { Node *n = makeNode(declChamps);
    addChild(n, makeNode(typeStruct));
    addChild(n, makeNode(id));
    addChild(n, $4);

    addChild($1, n);

    $$ = $1;
  }

  | TYPE Declarateurs ';' { Node *n = makeNode(declStruct);
    Node *param = makeNode(declChamps);

    if (strcmp($1, "int") == 0) addChild(param, makeNode(typeInt));
    else if (strcmp($1, "char") == 0) addChild(param, makeNode(typeChar));
    addChild(param, $2);

    addChild(n, param);
    $$ = n;
  }

  | STRUCT IDENT Declarateurs ';' { Node *n = makeNode(declStruct);
    Node *param = makeNode(declChamps);
    addChild(n, makeNode(typeStruct));
    addChild(n, makeNode(id));
    addChild(n, $3);

    addChild(n, param);
    $$ = n;
  }
;
```

Dans le corps de la déclaration d'un type structuré, on doit pouvoir déclarer des variables de type int ou char, ou bien des variables d'une structure. Par conséquent, on a ajouté **CorpsStruct** et **DeclStruct**, qui représentent les variables qui composent une structure. On a aussi fait en sorte qu'il faut toujours au moins un champ par structure.

```
EnTeteFonct:
  TYPE IDENT '(' Parametres ')' { Node *n = makeNode(enTete);
    if (strcmp($1, "int") == 0) addChild(n, makeNode(typeInt));
    else if (strcmp($1, "char") == 0) addChild(n, makeNode(typeChar));
    addChild(n, makeNode(id));
    if ($4 != NULL) addChild(n, $4);
    $$ = n;
  }
  | STRUCT IDENT IDENT '(' Parametres ')' { Node *n = makeNode(enTete);
    addChild(n, makeNode(typeStruct));
    addChild(n, makeNode(id));
    addChild(n, makeNode(id));
    if ($5 != NULL) addChild(n, $5);
    $$ = n;
  }
  | VOID IDENT '(' Parametres ')' { Node *n = makeNode(enTete);
    addChild(n, makeNode(typeVoid));
    addChild(n, makeNode(id));
    if ($4 != NULL) addChild(n, $4);
    $$ = n;
  }
;
```

Sachant qu'une fonction doit renvoyer un type structuré, et aussi en prendre en paramètres, on a donc fait le choix d'ajouter l'option du milieu, qui autorise les types structurés comme valeur de retour.

```
Parametres:
  VOID { Node *n = makeNode(Parametres);
    addChild(n, makeNode(typeVoid));
    $$ = n;
  }
  | ListTypVar { Node *n = makeNode(Parametres);
    addChild(n, $1);
    $$ = n;
  }
;
```

```

ListTypVar:
  ListTypVar ',' TYPE IDENT { Node *n = makeNode(TypVar);
                             if (strcmp($3, "int") == 0) addChild(n, makeNode(typeInt));
                             else if (strcmp($3, "char") == 0) addChild(n, makeNode(typeChar));
                             addChild(n, makeNode(id));

                             addChild($1, n);
                             $$ = $1;
                             }
  | ListTypVar ',' STRUCT IDENT IDENT { Node *n = makeNode(TypVar);
                                       addChild(n, makeNode(typeStruct));
                                       addChild(n, makeNode(id));
                                       addChild(n, makeNode(id));

                                       addChild($1, n);
                                       $$ = $1;
                                       }
  | TYPE IDENT { Node *n = makeNode(ListTypVar);
                Node *param = makeNode(TypVar);

                if (strcmp($1, "int") == 0) addChild(param, makeNode(typeInt));
                else if (strcmp($1, "char") == 0) addChild(param, makeNode(typeChar));
                addChild(param, makeNode(id));

                addChild(n, param);
                $$ = n;
                }
  | STRUCT IDENT IDENT { Node *n = makeNode(ListTypVar);
                        Node *param = makeNode(TypVar);

                        addChild(param, makeNode(typeStruct));
                        addChild(param, makeNode(id));
                        addChild(param, makeNode(id));

                        addChild(n, param);
                        $$ = n;
                        }
;

```

Par conséquent, la liste des paramètres est aussi modifiée.

Par exemple, si on a un type **aircraft**, une fonction peut avoir comme signature :

struct aircraft upgrade(struct aircraft a):

```

Instr:
  FieldAccess '=' Exp ';' { Node *n = makeNode(assign);
                           addChild(n, $1);
                           addChild(n, $3);
                           $$ = n;
                           }

```

```

F : ADDSUB F { Node *n;
    if ($1 == '+') n = makeNode(unaryplus);
    else          n = makeNode(unaryminus);
    addChild(n, $2);
    $$ = n;
}

| '!' F { Node *n = makeNode(notInstr);
    addChild(n, $2);
    $$ = n;
}

| '(' Exp ')' { $$ = $2; }
| NUM { $$ = makeNode(num); }
| CHARACTER { $$ = makeNode(character); }
| FieldAccess { $$ = $1; }
| IDENT '(' Arguments ')' { Node *n = makeNode(appelFonct);
    addChild(n, makeNode(id));
    addChild(n, $3);
    $$ = n;
}

;

```

```

FieldAccess:
    FieldAccess '.' IDENT { addChild($1, makeNode(id));
        $$ = $1;
    }

| IDENT { Node *n = makeNode(fieldAccess);
    addChild(n, makeNode(id));
    $$ = n;
}

;

```

Pour les instructions (affectations et opérations), il faut pouvoir accéder aux champs des structures, sous la forme : **structure.champ**. On a modifié les instructions, qui correspondent aux affectations, et la suite des instructions, où il y a les opérations.

FieldAccess a été créé, qui correspond soit aux champs des structures, soit à un simple identifiant.

Difficultés

On a rencontré plusieurs difficultés pendant le projet.

Tout d'abord, l'extension du langage n'a pas été difficile en soi, sachant que les types structurés en TPC sont simplifiés par rapport au C, et que les contraintes sont expliquées dans le sujet. Cependant, pour les accès aux champs, on a dû modifier la grammaire de base afin d'autoriser ceci (avec `FieldAccess`).

Au début, l'arbre abstrait a été plutôt compliqué à faire, mais c'est allé plus vite lorsqu'on a compris le concept.

Quand il fallait tester notre analyseur syntaxique dans le bac à sable sur e-learning, il y avait eu le **Test 9 - Core**, qui était le dernier test où notre analyseur ne renverrait pas la bonne valeur.

Bien qu'il y ait eu des tests incorrects (la majorité venant des tests sur les types structurés), on a bien effectué des modifications afin que ces tests passent. Au final, on est allé voir dans notre lexeur, on a effectivement vu ce qui n'allait pas, et enfin on a eu 100% des tests passés.

Tests

On a effectué deux jeux de tests, l'un avec des programmes TPC corrects, l'autre avec des programmes incorrects.

Pour lancer le script de déploiement de tests, depuis la racine du projet, il faut écrire :

```
python3 ./src/tests.py
```

Cela affichera les tests où le compilateur a eu faux, et le résultat final (les tests passés, le nombre de tests, et le pourcentage des tests qui sont passés).

Conclusion

La finalité de ce projet est de créer un analyseur syntaxique pour un langage précis, le langage TPC. Pour le faire on a utilisé Flex et Bison pour détecter les erreurs de syntaxes pour les programmes incorrects et pour créer un arbre pour les programmes qui fonctionnent.

Points positifs:

Phan:

- On avait déjà la base d'une grammaire.
- On en apprend plus, et on applique ce qui a été fait en TP.

Karl:

- C'est un projet intéressant qui nous permet d'avoir une idée plus claire sur le fonctionnement d'un analyseur syntaxique.

Points négatifs:

Phan:

- Le débogage n'était pas évident.
- Lorsque sur le sandbox, un test spécifique ne passait pas, et il fallait chercher l'erreur.

Karl:

- La difficulté rencontrée pour corriger l'affichage des arbres abstraits qui montraient des erreurs assez grosses à chaque fois.